

Curso Inicial Front-End

Clase 06: Box Model y Layouts

Módulo 1: Arquitectura y Maquetación Moderna (HTML y CSS)



Presentación

Todo elemento en la web es una caja. En esta clase dominaremos el **Modelo de Caja (Box Model)**, el concepto más crítico del diseño web: cómo se mide y cómo se espacian los elementos. Aprenderemos también las **propiedades de visualización (Display)**, que determinan si un elemento ocupa toda la fila o convive al lado de otros. Con estas bases, entenderemos el flujo del documento y cómo romperlo con posicionamiento.



Objetivos

- Dominar el Box Model: Content, Padding, Border y Margin.
- Comprender la diferencia entre `display: block` y `display: inline`.
- Aplicar posicionamiento **CSS** para colocar elementos con precisión.



Bloques Temáticos

- Tema 1: El modelo de caja: margin, padding, border, content.
 - Margin, padding, border, content.
- Tema 2: Propiedades de visualización (Display): Block vs Inline.
 - Display Block vs Inline.
 - Posicionamiento.
 - Z-Index.

En la clase anterior aprendimos los selectores, la cascada y la herencia. Ahora entraremos en el concepto fundamental que rige toda la maquetación: cada elemento del **HTML** es, físicamente, una caja rectangular invisible. Entender esa caja es entender el 80% del **CSS**.

Tema 1: El modelo de caja: margin, padding, border, content

1. Margin, padding, border, content

El "**Modelo de Caja**" en internet: Imagina que cada elemento es un cuadro.

En diseño web, todo lo que ves en una pantalla (un botón, una foto, un texto) es en realidad una **caja rectangular invisible**.

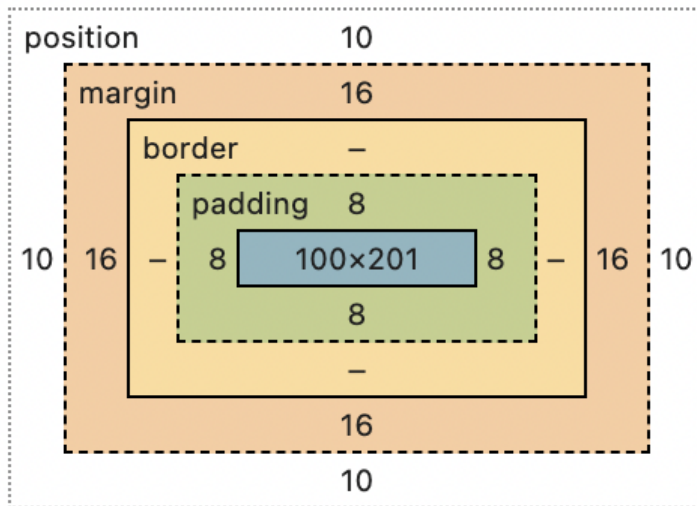
Para entender cómo se comporta, imagínalo como un cuadro colgado en una pared:

- **Content (El Lienzo):** Es el cuadro en sí, la pintura, el texto o la foto. Es el corazón de la caja y lo que define su tamaño original.
- **Padding (El Margen Interno):** Es el espacio en blanco (el paspartú) entre la pintura y el marco. Si le das más padding, el cuadro se hace más grande por dentro y respira mejor, pero mantiene su color de fondo.
- **Border (El Marco):** La línea física que envuelve el cuadro. Puede ser de madera gruesa, fina, de colores o directamente invisible, pero marca el límite real de tu elemento.
- **Margin (La distancia en la pared):** Es la zona de seguridad en la pared para que tu cuadro no choque con el cuadro de al lado. Es un espacio invisible y externo; no ensancha tu cuadro, solo empuja a los vecinos.



El resumen definitivo

- **Padding** es espacio hacia adentro (empuja el marco lejos del contenido).
- **Margin** es espacio hacia afuera (empuja a los demás elementos lejos del marco).



Esta imagen muestra el 'Modelo de Caja' real que usa un navegador web. En el centro en **azul** ves el **Lienzo** con sus medidas exactas. Alrededor, en **verde**, el **Paspartú (Padding)** agregando 8 píxeles de aire por cada lado. El **Marco (Border)** está representado en **amarillo** (con guiones, lo que significa que es invisible o no tiene grosor). Y finalmente, la **distancia en la pared (Margin)** en color **naranja**, que empuja a los vecinos a 16 píxeles de distancia.

Comparativa Vital: Margin vs Padding

Característica	padding (Relleno Interno)	margin (Margen Externo)
Ubicación	Dentro del borde.	Fuera del borde.
Color de Fondo	Hereda el background-color de la caja.	Siempre transparente.
Función Real	Empuja el contenido hacia adentro para que respire.	Empuja las cajas vecinas para que no choquen.

 **Checkpoint de Comprensión:** Tienes un botón azul que dice "Comprar". Querés que se vea más gordo con más espacio azul alrededor de la palabra. ¿Usás margin o padding?

El Reset Profesional y la Matemática de la Caja

```
/* El asterisco (*) selecciona absolutamente TODAS las etiquetas */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* Vital: el width incluye padding y border */
}
```

Con `box-sizing: border-box`, si declarás `width: 200px` con `padding: 20px`, la caja mide exactamente 200px. Sin él, mediría 240px (200 + 40 de padding). **Siempre usá este reset en la línea 1 de tu CSS.**

 **Recurso Oficial:** [MDN Web Docs - El Modelo de Caja](#)

Tema 2: Propiedades de visualización (Display): Block vs Inline.

1. Display Block vs Inline

La propiedad `display` determina cómo se comporta un elemento dentro del flujo del documento HTML.

Block: Ocupa toda la fila

Los elementos `block` siempre empiezan en una nueva línea y se extienden todo el ancho disponible de su contenedor, aunque su contenido sea pequeño.

- Ejemplos nativos: `<div>`, `<p>`, `<h1>` a `<h6>`, `<section>`, `<article>`
- Aceptan `width`, `height`, `margin` y `padding` en todas las direcciones.

```
.caja-block {  
  display: block;  
  width: 300px;  
  background-color: lightblue;  
  padding: 20px;  
  margin-bottom: 10px;  
}
```


Inline: Fluye dentro del texto

Los elementos `inline` no rompen el flujo del texto. Se colocan uno al lado del otro como palabras en una oración.

- Ejemplos nativos: `<span>`, `<a>`, `<strong>`, `<em>`, `<img>`
- **Limitación:** No aceptan `width` ni `height`. El `margin` y `padding` solo funcionan horizontalmente.

```
.etiqueta {  
  display: inline;  
  background-color: yellow;  
  padding: 2px 8px; /* solo horizontal funciona correctamente */  
}
```

Inline-Block: Lo mejor de ambos mundos

Se comporta inline (no rompe la línea) pero acepta `width`, `height` y `margin` en todas las direcciones.

```
.boton {  
  display: inline-block;  
  width: 120px;  
  height: 40px;  
  padding: 8px 16px;  
  background-color: blue;  
  color: white;  
}
```

Comparativa

Propiedad	block	inline	inline-block
¿Rompe línea?	Sí, siempre	No	No
Acepta width/height	Sí	No	Sí
Acepta margin vertical	Sí	No	Sí
Ejemplos nativos	div, p, h1	span, a	img, button

 **Checkpoint de Comprensión:** Tenés tres `<span>` con `display: inline`. Querés que cada uno tenga exactamente `width: 100px`. ¿Qué propiedad de display necesitás cambiar?

Mostrar y Ocultar Elementos: `display: none` vs `visibility: hidden`

Dos propiedades para ocultar elementos con comportamientos muy distintos:

Propiedad	¿Ocupa espacio en el layout?	¿Es visible?	Caso de uso típico
<code>display: none</code>	No. El elemento desaparece del flujo, como si no existiera.	No	Menús desplegables, modales, tabs
<code>visibility: hidden</code>	Sí. El espacio físico se reserva aunque no se vea.	No	Ocultar sin que el layout "salte"

```
/* El elemento desaparece y sus vecinos ocupan su lugar */
.modal { display: none; }
.modal.activo { display: block; }

/* El elemento es invisible pero su espacio queda reservado */
.placeholder { visibility: hidden; }
```

Tip profesional: En **JavaScript** usarás `element.style.display = "none"` y `element.style.display = "block"` constantemente para mostrar/ocultar partes de la UI. Es uno de los patrones DOM más frecuentes en la industria.

2. Posicionamiento

Por defecto, absolutamente todos los elementos en tu página nacen con una propiedad invisible llamada `position: static`. Para entender qué significa esto, imagina que tus etiquetas **HTML** son personas en la fila de un supermercado:

- Aparecen una debajo de la otra o al lado de la otra, estrictamente según el orden en que las escribiste en tu archivo **HTML**.
- **No puedes** moverlas dándoles coordenadas exactas en la pantalla.
- Ocupan un lugar físico sólido: nadie puede "encimarse" o flotar sobre la cabeza de otro.

Para romper esta fila aburrida y mover las cosas con total libertad geométrica, usaremos la propiedad `position` acompañada de coordenadas direccionales (`top`, `bottom`, `left`, `right`).



Recurso Oficial: [MDN Web Docs - Posicionamiento](https://developer.mozilla.org/es/docs/Web/CSS/position)

Posicionamiento Fuera de Flujo (Absolute): Pros y Contras

✓ Ventajas (Pros)	✗ Desventajas (Contras)
Libertad Total: Puedes colocar botones encima de videos o textos en cualquier esquina de la pantalla.	Caos Responsivo: Si abusas, al abrir la web en un celular todo se verá amontonado o cortado fuera de la pantalla.
Ideal para Popups/Modales: Es la herramienta obligatoria para crear alertas emergentes o menús desplegables superpuestos.	Rompe la Física (El Flujo): Como el elemento deja de ocupar su espacio real, los elementos de abajo "chocarán" subiendo hacia arriba, tapándose entre sí.

3. Tipos de Posicionamiento: ¡Saliendo de la fila!

Relative (Relativo): Un paso al costado

Al ponerle `position: relative` a una caja, se le otorga el permiso de moverse respecto a **dónde estaba su posición original** en la fila.

- **Lo clave:** Aunque tú lo muevas 50 píxeles a la derecha usando `left: 50px`, el espacio vacío que ocupaba originalmente sigue allí, como si dejara su fantasma. Las demás cajas de la fila no se acercan para ocupar su lugar.
- **Uso común:** Rara vez se usa para mover cajas. Se usa muchísimo para convertir a esa caja en el "Ancla" de sus cajas hijas (como veremos en el siguiente punto).

Absolute (Absoluto): El elemento que vuela libre

Al ponerle `position: absolute`, el elemento se "sale totalmente del flujo de la página". Es como si despegara de la fila y empezara a flotar por encima de los demás.

- **Lo clave:** Ya no ocupa espacio físico en la hoja. Las cajas de abajo saltarán hacia arriba ocupando su lugar como si él hubiera dejado de existir.
- **¿Respecto a qué se mueve? El Algoritmo de Búsqueda** 🕵️

- Cuando le dices `top: 0`, el navegador no sabe exactamente de qué techo está hablando. Así que busca hacia arriba, entre los contenedores padres de esa caja flotante.
- Busca al **primer ancestro** que NO sea `static` (es decir, el primer padre que sea `relative` o `absolute`).
- Si no encuentra a ningún padre con posicionamiento, se desespera y se posiciona respecto al techo de **toda la pantalla de Google Chrome** (`<body>`).

🛑 **Checkpoint de Comprensión:** Tienes una caja "Padre" gigante, y adentro tienes un botón "Hijo". Al hijo le pones `position: absolute` y `bottom: 0`. Tu objetivo es que el hijo se quede pegado al suelo **solo de la caja Padre**. Si se te olvida ponerle `position: relative` a la caja Padre, ¿qué pasará con el botón Hijo? (Pista: Lee el "Algoritmo de búsqueda" arriba).

Fixed (Fijo): La cámara de seguridad inamovible

El elemento se sale del flujo y se queda pegado eternamente a las coordenadas de la **ventana física del monitor**.

- **Lo clave:** No importa cuánto gires la rueda del ratón (scroll) hacia abajo, el elemento jamás desaparecerá de tu vista. Es ideal para la barra de navegación superior (Navbar) permanente o el clásico botón circular de WhatsApp flotante abajo a la derecha.

Sticky (Pegajoso): El imán inteligente

Es un híbrido fascinante. Se comporta como un elemento normal en la fila hasta que el usuario hace scroll hacia abajo y la caja toca el límite que tú le marcaste (ej: `top: 0`). En ese instante, deja de subir y se queda "pegado" al techo del monitor como un imán mientras sigas scrolleando hacia abajo.

4. Z-Index

Al sacar elementos del flujo (`absolute` o `fixed`), estos empezarán a montarse unos sobre otros. Para decidir quién aplasta a quién, usamos la propiedad `z-index`, que funciona como un sistema de pisos en un edificio (quien tenga el número más alto, va más arriba).

¿Alguna vez pusiste un enorme `z-index: 9999` intentando que tu botón pase al frente, pero se quedó aplastado abajo igual? Eso pasa por ignorar el **Contexto de Apilamiento**.

- **Requisito Vital:** `z-index` solo funciona si el elemento tiene un `position` configurado (`relative`, `absolute` o `fixed`). ¡Jamás funcionará en un `static`!
- **La regla de la prisión local:** Si un Padre tiene `z-index: 1`, todos sus hijos están prisioneros en ese "Piso 1" del edificio.
- Aunque un Hijo tenga adentro un `z-index: 9999`, nunca podrá estar por encima visualmente de otra caja vecina Padre que esté en el "Piso 2" (`z-index: 2`). El `z-index` no es global para toda la web, está limitado por el nivel de `z-index` de su contenedor padre.

Documentación Oficial (Referencias)

- **CSS:** <https://developer.mozilla.org/es/docs/Web/CSS>
- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>